

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_634e841e-51b\agent-a25040250b7f7d198.jsonl`
- SHA-256: `b82484c3d206af39cd526a732db3a69e533cbf5537b99f9874e1ca1a30697841`
- Source modified: `2026-07-06T08:41:48+00:00`
- Imported at: `2026-07-08T16:00:06+00:00`
- project: `wf_634e841e-51b`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T08:39:27.319Z)

Reviewing GitHub PR #38 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-d-command-service -> main, one commit 22abe72).

It implements ADR 0011 Track D: a REFACTOR that extracts command parsing + config-mutation logic from the

Telethon adapter into a provider-independent CommandService. THE PRIMARY REVIEW RISK IS BEHAVIORAL

EQUIVALENCE: the new code must behave identically to the old inline logic for every command.

Changed files ONLY: src/tg_video_filter/command_service.py (NEW),
src/tg_video_filter/commands.py (refactored
407->~120 line thin Telethon adapter), tests/test_command_service.py (NEW, 20 tests). Diff
+650/-376.
tests/test_commands.py is UNCHANGED and still passes (backwards-compat surface).

To compare against the pre-refactor behaviour, read the OLD file:

```
git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/commands.py
```

The new logic lives in command_service.py (CommandService.handle + parsers + formatters) and the new

commands.py delegates to it. Key already-checked facts (verify independently, don't just trust):

- regex patterns are byte-identical old vs new; branch order is the same.
- old handler did text.rstrip() then per-branch logic; new handler passes raw text and handle() does text.rstrip().
- old _update_config RELOADED cfg from db each call; new CommandService._update uses the passed current_cfg,
and the new adapter loads current_cfg = db.load_filter_config(...) fresh right before handle(). Assess equivalence.
- both old and new use cfg.model_copy(update=...) which SKIPS pydantic validation (pre-existing, not new).

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
- A STRONG equivalence check: feed a battery of command strings through BOTH the new CommandService.handle()
and (on a checkout of main, or by reasoning from the old source) the old handler, and diff
reply_text +
resulting saved FilterConfig. Report ANY divergence (different reply string, different config
mutation,
different validation threshold, different branch taken, different order).
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check clean; ruff format clean;
pytest
482 passed @ 97.05% (floor 80%, up from 96.52%).
- Report ONLY defects in THIS PR's diff. For a refactor, a BEHAVIORAL DIVERGENCE from the old

code is the

highest-value finding. Also flag: lost functionality, broken re-exports, new bugs in extracted logic.

No praise. Concrete failure scenario per finding. Rank most-severe first. Honest severity.

- Severity: blocker (wrong behaviour users hit / lost command), high (real divergence, narrow trigger),

medium (edge divergence or latent), low (hygiene/style).

DIMENSION: test quality & coverage.

Investigate: test_command_service.py (20 tests) ? do they cover the failure/edge branches

(below-min, malformed,

help fallbacks, noise) AND assert the actual reply strings + config mutations, or just smoke-test? Is the new

adapter on_command (make_config_command_handler) still covered end-to-end (out/peer gate, reply, engine reload)

now that logic moved out ? or did handler coverage drop? Does any old test_commands.py assertion

now pass only

because of a re-export shim rather than real behaviour? Coverage rose to 97.05% ? any NEW

uncovered lines in

command_service.py / commands.py? Return findings per schema.

2. assistant (2026-07-06T08:39:32.129Z)

I'll start by reading the relevant files and comparing old vs new behavior.

3. user (2026-07-06T08:39:34.374Z)

```
"""Config commands: /set size, /set length, /show config.
```

These commands are sent to the user's own Saved Messages (like /backfill) and update the persisted ``FilterConfig``. The filter engine is hot-reloaded so the new config takes effect immediately without a restart.

See ``docs/adr/0007-inline-config-commands.md`` for the design rationale.
"""

```
from __future__ import annotations
```

```
import logging
```

```
import re
```

```
from collections.abc import Callable
```

```
import aiosqlite
```

```
from telethon import TelegramClient
```

```
from telethon.errors import RPCError
```

```
from tg_video_filter import db
```

```
from tg_video_filter.filters import FilterEngine
```

```
from tg_video_filter.models import FilterConfig, User
```

```
log = logging.getLogger(__name__)
```

Key into the shared engine_holder dict. Dict swap is atomic under the GIL for single-threaded asyncio dispatch (see ADR 0007).

```
ENGINE_KEY = "engine"
```

--- regex patterns -----

```
_SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
```

Matches both "/set length" and "/set max length" (legacy alias).

Kept separate from _MIN_LENGTH_RE for clarity; patterns are mutually exclusive.

```

_LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
_MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
_MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)
_SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
_SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
_EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)

```

Catch-all: matches any "/set exclude ..." that didn't match the strict pattern, so we can reply with a friendly usage message instead of silence.

```

_EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)

```

Catch-all for malformed /set min resolution|res (same pattern as _EXCLUDE_HELP_RE)

```

_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)

```

--- unit parsing -----

```

def parse_size_mb(value_str: str, unit: str) -> int:
    """Parse a size value + unit string into mebibytes.

    >>> parse_size_mb("10", "MB")
    10
    >>> parse_size_mb("1", "GB")
    1024
    >>> parse_size_mb("500", "KB")
    1

    Note: returns the raw arithmetic result; callers must enforce
    ``FilterConfig`` domain constraints (e.g. ``max_size_mb >= 1``).
    """
    value = int(value_str)
    unit = unit.lower().rstrip("b") # "MB" ? "m", "KB" ? "k", "GB" ? "g"
    multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
    if unit in multipliers:
        raw = value * multipliers[unit]
        if unit == "k":
            raw = max(1, raw // 1024)
        return raw
    raise ValueError(f"Unknown size unit: {unit}")

```

```

def parse_duration_s(value_str: str, unit: str) -> int:
    """Parse a duration value + unit string into seconds.

    >>> parse_duration_s("15", "m")
    900
    >>> parse_duration_s("2", "h")
    7200
    >>> parse_duration_s("90", "s")
    90

    Note: returns the raw arithmetic result; callers must enforce
    ``FilterConfig`` domain constraints (e.g. max >= min).
    """
    value = int(value_str)
    unit = unit.lower()
    multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
    if unit in multipliers:
        return value * multipliers[unit]
    raise ValueError(f"Unknown time unit: {unit}")

```

--- formatting helpers -----

```

def _format_duration(seconds: int | None) -> str:
    """Format seconds as a human-readable duration string."""
    if seconds is None:
        return "unlimited"
    if seconds == 0:
        return "0s"
    if seconds < 60:
        return f"{seconds}s"
    if seconds < 3600:
        m, s = divmod(seconds, 60)
        return f"{m}m{s}s" if s else f"{m}m"
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    parts = f"{h}h"
    if m:
        parts += f"{m}m"
    if s:
        parts += f"{s}s"
    return parts

def _format_size(mb: int | None) -> str:
    """Format mebibytes as a human-readable size string."""
    if mb is None:
        return "unlimited"
    if mb == 0:
        return "0MB"
    if mb >= 1024:
        gb = mb / 1024
        if gb == int(gb):
            return f"{int(gb)}GB"
        return f"{gb:.1f}GB"
    return f"{mb}MB"

def format_config(cfg: FilterConfig) -> str:
    """Format a ``FilterConfig`` as a readable Telegram message."""
    lines = [
        "? **Current Filter Configuration**",
        "",
        f"? **Duration:** {_format_duration(cfg.min_duration_s)} - "
        f"{_format_duration(cfg.max_duration_s)}",
        f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if cfg.target_height else "disabled"}",
        f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
        f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
        f"? **Delivery target:** `{cfg.delivery_target}`",
        "",
        "? **Content exclusions:**",
        f"    GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
        f"    Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
        f"    Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
    ]
    return "\n".join(lines)

def format_commands() -> str:
    """Return a help message listing all available commands."""
    return "\n".join(
        [
            "? **Available Commands**",
            "",
            "? **Filter config**",
            "/show config ? Show current filter settings",
            "/set size <N><unit> ? Set max file size (e.g. /set size 10MB)",
        ]
    )

```

```

    "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing `B`",
    "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
    "    or `/set max length <N><unit>` ? same as above",
    "    Units: `s` (default), `m`, `h` ? case-insensitive",
    "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length 5s`)",
    "    Units: `s` (default), `m`, `h` ? case-insensitive",
    "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution
720p`)",
    "    or `/set min res <N>p` ? same as above",
    "",
    "? **Content exclusions** (all off by default)",
    "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
    "`/set exclude round on|off` ? Exclude circular video messages",
    "`/set exclude nosound on|off` ? Exclude videos without audio\n"
    "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
    "",
    "? **Backfill**",
    "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default
90d)",
    "`/stop` ? Cancel running backfill",
    "",
    "\u2139\u2013 **Help**",
    "`/show commands` ? Show this help message",
]
)

```

--- command handler factory -----

```

def make_config_command_handler(
    user: User,
    conn: aiosqlite.Connection,
    engine_holder: dict[str, FilterEngine],
) -> Callable:
    """Return an async handler for /set and /show config commands.

    The handler only responds to messages the user sends to themselves
    (Saved Messages). It persists config changes via ``db.save_filter_config``
    and hot-reloads the filter engine through ``engine_holder``.

    ``engine_holder`` is a mutable dict that both the live-message handler and
    the backfill handler read from, so a config change is picked up by all
    components immediately.
    """

    async def on_command(event: object) -> None:
        message = getattr(event, "message", None)
        if message is None:
            return

        # Only respond to messages the user sent to themselves.
        if not getattr(message, "out", False):
            return
        peer = getattr(message, "peer_id", None)
        peer_user_id = getattr(peer, "user_id", None) if peer else None
        if peer_user_id != user.id:
            return

        text: str = getattr(message, "message", "") or ""
        # Strip trailing whitespace so mobile keyboards don't silently
        # reject commands (e.g. "/show config " ? "/show config").
        text = text.rstrip()
        client: TelegramClient = getattr(event, "client", None) # type: ignore[assignment]

```

```

# --- /set size <value><unit> -----
m = _SIZE_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_mb = parse_size_mb(value_str, unit)
    except ValueError:
        await _reply(client, message, "? Invalid size format. Use e.g. `/set size
10MB`.")
        return
    if new_mb < 1:
        await _reply(
            client, message, "? Size must be at least 1 MB (`FilterConfig` constraint).")
        return
    cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max file size set to **{_format_size(new_mb)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set length <value><unit> -----
m = _LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_secs = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client, message, "? Invalid duration format. Use e.g. `/set length 15m`."
        )
        return
    # Enforce max_duration_s >= min_duration_s so every video isn't
    # silently rejected (and so the FilterConfig validator passes on
    # next startup).
    current_cfg = await db.load_filter_config(conn, user.id)
    min_secs = current_cfg.min_duration_s or 0
    if new_secs < min_secs:
        await _reply(
            client,
            message,
            f"? Max duration ({_format_duration(new_secs)}) must be "
            f">= min duration ({_format_duration(min_secs)}).",
        )
        return
    cfg = await _update_config(conn, user.id, max_duration_s=new_secs)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max duration set to **{_format_duration(new_secs)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min length <value><unit> -----
m = _MIN_LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_min = parse_duration_s(value_str, unit)

```

```

except ValueError:
    await _reply(
        client,
        message,
        "? Invalid duration format. Use e.g. `/set min length 5s`.",
    )
    return
# Enforce min_duration_s <= max_duration_s.
current_cfg = await db.load_filter_config(conn, user.id)
max_secs = current_cfg.max_duration_s or 0
if max_secs > 0 and new_min > max_secs:
    await _reply(
        client,
        message,
        f"? Min duration ({_format_duration(new_min)}) must be "
        f"<= max duration ({_format_duration(max_secs)}).",
    )
    return
cfg = await _update_config(conn, user.id, min_duration_s=new_min)
engine_holder[ENGINE_KEY] = FilterEngine(cfg)
await _reply(
    client,
    message,
    f"? Min duration set to **{_format_duration(new_min)}**."
    f"\n\n{format_config(cfg)}",
)
return

# --- /set min resolution <N>p -----
m = _MIN_RES_RE.match(text)
if m:
    new_height = int(m.group(1))
    # Pre-validate for a friendly message (FilterConfig also enforces ge=1).
    if new_height < 1:
        await _reply(client, message, "? Resolution must be at least 1p.")
        return
    cfg = await _update_config(conn, user.id, target_height=new_height)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min resolution set to **{new_height}p**." f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for malformed /set min resolution|res.
if _MIN_RES_HELP_RE.match(text):
    await _reply(
        client,
        message,
        "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)",
    )
    return

# --- /set exclude <gif|round|nosound> <on|off> -----
m = _EXCLUDE_RE.match(text)
if m:
    what = m.group(1).lower()
    state = m.group(2).lower()
    enabled = state == "on"
    labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
    field_map = {
        "gif": "exclude_gif",
        "round": "exclude_round",

```

```

        "nosound": "exclude_nosound",
    }
    cfg = await _update_config(conn, user.id, **{field_map[what]: enabled})
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for invalid /set exclude (e.g. "gif true", missing on/off).
if _EXCLUDE_HELP_RE.match(text):
    await _reply(
        client,
        message,
        "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
        "Example: `/set exclude gif on`",
    )
    return

# --- /show config -----
if _SHOW_CONFIG_RE.match(text):
    cfg = await db.load_filter_config(conn, user.id)
    await _reply(client, message, format_config(cfg))
    return

# --- /show commands -----
if _SHOW_COMMANDS_RE.match(text):
    await _reply(client, message, format_commands())
    return

return on_command

```

--- internal helpers -----

```

async def _update_config(
    conn: aiosqlite.Connection,
    user_id: int,
    max_size_mb: int | None = None,
    max_duration_s: int | None = None,
    min_duration_s: int | None = None,
    target_height: int | None = None,
    exclude_gif: bool | None = None,
    exclude_round: bool | None = None,
    exclude_nosound: bool | None = None,
) -> FilterConfig:
    """Load the current config, apply partial updates, persist, and return."""
    cfg = await db.load_filter_config(conn, user_id)
    if max_size_mb is not None:
        cfg = cfg.model_copy(update={"max_size_mb": max_size_mb})
    if max_duration_s is not None:
        cfg = cfg.model_copy(update={"max_duration_s": max_duration_s})
    if min_duration_s is not None:
        cfg = cfg.model_copy(update={"min_duration_s": min_duration_s})
    if target_height is not None:
        cfg = cfg.model_copy(update={"target_height": target_height})
    if exclude_gif is not None:
        cfg = cfg.model_copy(update={"exclude_gif": exclude_gif})
    if exclude_round is not None:
        cfg = cfg.model_copy(update={"exclude_round": exclude_round})

```



```

if exclude_nosound is not None:
    cfg = cfg.model_copy(update={"exclude_nosound": exclude_nosound})
await db.save_filter_config(conn, user_id, cfg)
log.info(
    "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
    "target_height=%s "
    "exclude_gif=%s exclude_round=%s exclude_nosound=%s",
    user_id,
    cfg.max_size_mb,
    cfg.min_duration_s,
    cfg.max_duration_s,
    cfg.target_height,
    cfg.exclude_gif,
    cfg.exclude_round,
    cfg.exclude_nosound,
)
return cfg

async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
    """Send a reply, logging network errors but not raising.

    Only swallows network/RPC errors so programming bugs (KeyError,
    AttributeError, TypeError) still surface loudly during development.
    """
    try:
        await client.send_message("me", text)
    except (OSError, TimeoutError, RPCError):
        log.warning("config_reply_failed", exc_info=True)

__all__ = [
    "ENGINE_KEY",
    "_EXCLUDE_RE",
    "_LENGTH_RE",
    "_MIN_LENGTH_RE",
    "_MIN_RES_RE",
    "_MIN_RES_HELP_RE",
    "_SHOW_COMMANDS_RE",
    "_SHOW_CONFIG_RE",
    "_SIZE_RE",
    "_format_duration",
    "_format_size",
    "_update_config",
    "format_commands",
    "format_config",
    "make_config_command_handler",
    "parse_duration_s",
    "parse_size_mb",
]

```

4. user (2026-07-06T08:39:34.380Z)

```

1     """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3     Extracts command logic from the Telethon adapter (`commands.py`) so the
4     same `/set`, `/show` commands work identically through a Bot API
5     front-door. The service is pure: strings in, `CommandResult` out ? no
6     Telethon objects, no network calls.
7
8     See `docs/adr/0007-inline-config-commands.md` for the command model and
9     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API motivation.
10    """
11
12    from __future__ import annotations

```

```

13
14 import logging
15 import re
16 from dataclasses import dataclass
17
18 import aiosqlite
19
20 from tg_video_filter import db
21 from tg_video_filter.models import FilterConfig
22
23 log = logging.getLogger(__name__)
24
25 # Key into the shared engine_holder dict (imported by commands.py's adapter).
26 ENGINE_KEY = "engine"
27
28 # --- regex patterns -----
29
30 _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmg]?b?)$", re.IGNORECASE)
31 _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)", re.IGNORECASE)
32 _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)", re.IGNORECASE)
33 _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34 _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35 _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36 _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37 _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38 _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41 # --- unit parsing -----
42
43
44 def parse_size_mb(value_str: str, unit: str) -> int:
45     """Parse a size value + unit string into mebibytes."""
46     value = int(value_str)
47     unit = unit.lower().rstrip("b")
48     multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
49     if unit in multipliers:
50         raw = value * multipliers[unit]
51         if unit == "k":
52             raw = max(1, raw // 1024)
53         return raw
54     raise ValueError(f"Unknown size unit: {unit}")
55
56
57 def parse_duration_s(value_str: str, unit: str) -> int:
58     """Parse a duration value + unit string into seconds."""
59     value = int(value_str)
60     unit = unit.lower()
61     multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
62     if unit in multipliers:
63         return value * multipliers[unit]
64     raise ValueError(f"Unknown time unit: {unit}")
65
66
67 # --- formatting helpers -----
68
69
70 def _format_duration(seconds: int | None) -> str:
71     if seconds is None:
72         return "unlimited"
73     if seconds == 0:
74         return "0s"
75     if seconds < 60:

```

```

76         return f"{seconds}s"
77     if seconds < 3600:
78         m, s = divmod(seconds, 60)
79         return f"{m}{s}s" if s else f"{m}m"
80     h, rem = divmod(seconds, 3600)
81     m, s = divmod(rem, 60)
82     parts = f"{h}h"
83     if m:
84         parts += f"{m}m"
85     if s:
86         parts += f"{s}s"
87     return parts
88
89
90     def _format_size(mb: int | None) -> str:
91         if mb is None:
92             return "unlimited"
93         if mb == 0:
94             return "0MB"
95         if mb >= 1024:
96             gb = mb / 1024
97             if gb == int(gb):
98                 return f"{int(gb)}GB"
99             return f"{gb:.1f}GB"
100         return f"{mb}MB"
101
102
103     def format_config(cfg: FilterConfig) -> str:
104         """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
105         lines = [
106             "? **Current Filter Configuration**",
107             "",
108             f"? **Duration:** {_format_duration(cfg.min_duration_s)} - "
109             f"{_format_duration(cfg.max_duration_s)}",
110             f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if "
111             f"cfg.target_height else 'disabled'}",
112             f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
113             f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
114             f"? **Delivery target:** `{cfg.delivery_target}`",
115             "",
116             "? **Content exclusions:**",
117             f"    GIF/animated: {' excluded' if cfg.exclude_gif else '? allowed'}",
118             f"    Round videos: {' excluded' if cfg.exclude_round else '? allowed'}",
119             f"    Silent (no audio): {' excluded' if cfg.exclude_nosound else '? allowed'}",
120         ]
121         return "\n".join(lines)
122
123     def format_commands() -> str:
124         """Return a help message listing all available commands."""
125         return "\n".join(
126             [
127                 "? **Available Commands**",
128                 "",
129                 "? **Filter config**",
130                 "`/show config` ? Show current filter settings",
131                 "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
132                 "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing "
133                 "`B`",
134                 "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
135                 "    or `/set max length <N><unit>` ? same as above",
136                 "    Units: `s` (default), `m`, `h` ? case-insensitive",
137                 "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length "
138                 "5s`)",
139                 "    Units: `s` (default), `m`, `h` ? case-insensitive",

```

```

137         "`/set min resolution <N>p` ? Set min video height (e.g. `/set min
resolution 720p`)",
138         "    or `/set min res <N>p` ? same as above",
139         "",
140         "? **Content exclusions** (all off by default)",
141         "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
142         "`/set exclude round on|off` ? Exclude circular video messages",
143         "`/set exclude nosound on|off` ? Exclude videos without audio\n"
144         "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
too.",
145         "",
146         "? **Backfill**",
147         "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
(default 90d)",
148         "`/stop` ? Cancel running backfill",
149         "",
150         "\u2139ufe0f **Help**",
151         "`/show commands` ? Show this help message",
152     ]
153 )
154
155
156 # --- command result -----
157
158
159 @dataclass
160 class CommandResult:
161     """The output of executing a command ? provider-independent.
162
163     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
164     and ``config_changed`` to decide whether to hot-reload the filter engine.
165     """
166
167     reply_text: str = ""
168     config_changed: bool = False
169
170
171 # --- command service -----
172
173
174 class CommandService:
175     """Provider-independent command execution.
176
177     Parses a command string against an account's current config and DB,
178     returning a ``CommandResult``. No Telethon objects involved ? pure string
179     in, effects out. The Telethon adapter (``commands.py``) handles the
180     transport (checking message.out, sending the reply); the Bot API adapter
181     (future) calls ``handle()`` with the ``from.id`` from the update.
182
183     Usage::
184
185         svc = CommandService()
186         result = await svc.handle(user_id, text, conn, current_cfg)
187         if result.config_changed:
188             engine_holder["engine"] = FilterEngine(current_cfg)
189         await send_reply(result.reply_text)
190     """
191
192 # -- public API -----
193
194 async def handle(
195     self,
196     user_id: int,
197     text: str,
198     conn: aisqlite.Connection,

```

```

199         current_cfg: FilterConfig,
200     ) -> CommandResult:
201         """Parse and execute a command text, returning the reply + change flag."""
202         text = text.rstrip()
203
204         # --- /set size <value><unit> -----
205         m = _SIZE_RE.match(text)
206         if m:
207             value_str, unit = m.group(1), m.group(2) or ""
208             try:
209                 new_mb = parse_size_mb(value_str, unit)
210             except ValueError:
211                 return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
212             if new_mb < 1:
213                 return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
214             cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
215             return CommandResult(
216                 f"? Max file size set to
**[_format_size(new_mb)]*.\\n\\n{format_config(cfg)}",
217                 config_changed=True,
218             )
219
220         # --- /set length <value><unit> -----
221         m = _LENGTH_RE.match(text)
222         if m:
223             value_str, unit = m.group(1), m.group(2) or ""
224             try:
225                 new_secs = parse_duration_s(value_str, unit)
226             except ValueError:
227                 return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
228             min_secs = current_cfg.min_duration_s or 0
229             if new_secs < min_secs:
230                 return CommandResult(
231                     f"? Max duration ({_format_duration(new_secs)}) must be "
232                     f">= min duration ({_format_duration(min_secs)})."
233                 )
234             cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
235             return CommandResult(
236                 f"? Max duration set to
**[_format_duration(new_secs)]*.\\n\\n{format_config(cfg)}",
237                 config_changed=True,
238             )
239
240         # --- /set min length <value><unit> -----
241         m = _MIN_LENGTH_RE.match(text)
242         if m:
243             value_str, unit = m.group(1), m.group(2) or ""
244             try:
245                 new_min = parse_duration_s(value_str, unit)
246             except ValueError:
247                 return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
248             max_secs = current_cfg.max_duration_s or 0
249             if max_secs > 0 and new_min > max_secs:
250                 return CommandResult(
251                     f"? Min duration ({_format_duration(new_min)}) must be "
252                     f"<= max duration ({_format_duration(max_secs)})."
253                 )
254             cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
255             return CommandResult(
256                 f"? Min duration set to

```

```

**[_format_duration(new_min)]**.\n\n{format_config(cfg)}",
257         config_changed=True,
258     )
259
260     # --- /set min resolution <N>p -----
261     m = _MIN_RES_RE.match(text)
262     if m:
263         new_height = int(m.group(1))
264         if new_height < 1:
265             return CommandResult("? Resolution must be at least 1p.")
266         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
267         return CommandResult(
268             f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
269             config_changed=True,
270         )
271
272     if _MIN_RES_HELP_RE.match(text):
273         return CommandResult(
274             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
275         )
276
277     # --- /set exclude <gif|round|nosound> <on|off> -----
278     m = _EXCLUDE_RE.match(text)
279     if m:
280         what = m.group(1).lower()
281         state = m.group(2).lower()
282         enabled = state == "on"
283         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
284         field_map = {
285             "gif": "exclude_gif",
286             "round": "exclude_round",
287             "nosound": "exclude_nosound",
288         }
289         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
290         return CommandResult(
291             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
292             f"\n\n{format_config(cfg)}",
293             config_changed=True,
294         )
295
296     if _EXCLUDE_HELP_RE.match(text):
297         return CommandResult(
298             "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
299             "Example: `/set exclude gif on`"
300         )
301
302     # --- /show config -----
303     if _SHOW_CONFIG_RE.match(text):
304         cfg = await db.load_filter_config(conn, user_id)
305         return CommandResult(format_config(cfg))
306
307     # --- /show commands -----
308     if _SHOW_COMMANDS_RE.match(text):
309         return CommandResult(format_commands())
310
311     # Not a recognised command ? return empty result (noise).
312     return CommandResult()
313
314     # -- internal -----
315
316     async def _update(
317         self,

```

```

318         conn: aiosqlite.Connection,
319         user_id: int,
320         current: FilterConfig,
321         max_size_mb: int | None = None,
322         max_duration_s: int | None = None,
323         min_duration_s: int | None = None,
324         target_height: int | None = None,
325         exclude_gif: bool | None = None,
326         exclude_round: bool | None = None,
327         exclude_nosound: bool | None = None,
328     ) -> FilterConfig:
329         """Apply partial updates to the account's config, persist, and return."""
330         updates: dict[str, object] = {}
331         if max_size_mb is not None:
332             updates["max_size_mb"] = max_size_mb
333         if max_duration_s is not None:
334             updates["max_duration_s"] = max_duration_s
335         if min_duration_s is not None:
336             updates["min_duration_s"] = min_duration_s
337         if target_height is not None:
338             updates["target_height"] = target_height
339         if exclude_gif is not None:
340             updates["exclude_gif"] = exclude_gif
341         if exclude_round is not None:
342             updates["exclude_round"] = exclude_round
343         if exclude_nosound is not None:
344             updates["exclude_nosound"] = exclude_nosound
345         cfg = current.model_copy(update=updates) if updates else current
346         await db.save_filter_config(conn, user_id, cfg)
347         log.info(
348             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
349             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
350             user_id,
351             cfg.max_size_mb,
352             cfg.min_duration_s,
353             cfg.max_duration_s,
354             cfg.target_height,
355             cfg.exclude_gif,
356             cfg.exclude_round,
357             cfg.exclude_nosound,
358         )
359         return cfg
360
361
362     __all__ = [
363         "ENGINE_KEY",
364         "CommandResult",
365         "CommandService",
366         "_format_duration",
367         "_format_size",
368         "format_config",
369         "format_commands",
370         "parse_duration_s",
371         "parse_size_mb",
372     ]
373

```

5. user (2026-07-06T08:39:34.637Z)

```

1     """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2
3     This module is a thin Telethon adapter. It handles Telethon-specific transport
4     (checking `message.out`, `peer_id`, sending replies) and delegates command
5     parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6

```

```

7      Re-exports regex patterns and unit-parsers for backwards-compatibility with
8      existing tests (`tests/test_commands.py`).
9
10     See `docs/adr/0007-inline-config-commands.md` for the command model and
11     `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12     """
13
14     from __future__ import annotations
15
16     import logging
17     from collections.abc import Callable
18
19     import aiosqlite
20     from telethon import TelegramClient
21     from telethon.errors import RPCError
22
23     from tg_video_filter import db
24     from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25     from tg_video_filter.filters import FilterEngine
26     from tg_video_filter.models import FilterConfig, User
27
28     log = logging.getLogger(__name__)
29
30     from tg_video_filter.command_service import ( # noqa: E402
31         _EXCLUDE_HELP_RE,
32         _EXCLUDE_RE,
33         _LENGTH_RE,
34         _MIN_LENGTH_RE,
35         _MIN_RES_HELP_RE,
36         _MIN_RES_RE,
37         _SHOW_COMMANDS_RE,
38         _SHOW_CONFIG_RE,
39         _SIZE_RE,
40         _format_duration,
41         _format_size,
42         format_commands,
43         format_config,
44         parse_duration_s,
45         parse_size_mb,
46     )
47
48
49     def make_config_command_handler(
50         user: User,
51         conn: aiosqlite.Connection,
52         engine_holder: dict[str, FilterEngine],
53     ) -> Callable:
54         svc = CommandService()
55
56         async def on_command(event: object) -> None:
57             message = getattr(event, "message", None)
58             if message is None:
59                 return
60             if not getattr(message, "out", False):
61                 return
62             peer = getattr(message, "peer_id", None)
63             peer_user_id = getattr(peer, "user_id", None) if peer else None
64             if peer_user_id != user.id:
65                 return
66
67             text: str = getattr(message, "message", "") or ""
68             client: TelegramClient = getattr(event, "client", None) # type:
69             ignore[assignment]
70
71             current_cfg = await db.load_filter_config(conn, user.id)

```



```

71         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
72
73     if result.config_changed:
74         new_cfg = await db.load_filter_config(conn, user.id)
75         engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
76
77     if result.reply_text:
78         await _reply(client, message, result.reply_text)
79
80     return on_command
81
82
83 # --- backwards-compat helpers (test_commands.py references) -----
84
85
86 async def _update_config(
87     conn: aiosqlite.Connection,
88     user_id: int,
89     max_size_mb: int | None = None,
90     max_duration_s: int | None = None,
91     min_duration_s: int | None = None,
92     target_height: int | None = None,
93     exclude_gif: bool | None = None,
94     exclude_round: bool | None = None,
95     exclude_nosound: bool | None = None,
96 ) -> FilterConfig:
97     """Deprecated wrapper ? delegates to ``CommandService._update``.
98
99     Kept for backwards-compatibility with ``tests/test_commands.py`` which
100     imports ``_update_config`` from here. New code should call
101     :meth:`CommandService.handle` which returns a :class:`CommandResult`.
102     """
103     svc = CommandService()
104     cfg = await db.load_filter_config(conn, user_id)
105     return await svc._update(
106         conn,
107         user_id,
108         cfg,
109         max_size_mb=max_size_mb,
110         max_duration_s=max_duration_s,
111         min_duration_s=min_duration_s,
112         target_height=target_height,
113         exclude_gif=exclude_gif,
114         exclude_round=exclude_round,
115         exclude_nosound=exclude_nosound,
116     )
117
118
119 async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
120     try:
121         await client.send_message("me", text)
122     except (OSError, TimeoutError, RPCError):
123         log.warning("config_reply_failed", exc_info=True)
124
125
126 __all__ = [
127     "ENGINE_KEY",
128     "_EXCLUDE_RE",
129     "_LENGTH_RE",
130     "_MIN_LENGTH_RE",
131     "_MIN_RES_RE",
132     "_SHOW_CONFIG_RE",
133     "_SHOW_COMMANDS_RE",
134     "_EXCLUDE_HELP_RE",
135     "_MIN_RES_HELP_RE",

```

```

136         "_SIZE_RE",
137         "_format_duration",
138         "_format_size",
139         "_update_config",
140         "format_commands",
141         "format_config",
142         "make_config_command_handler",
143         "parse_duration_s",
144         "parse_size_mb",
145     ]
146

```

6. user (2026-07-06T08:39:35.219Z)

```

1     """Tests for the provider-independent CommandService (command_service.py).
2
3     These tests exercise command parsing and config mutation without any
4     Telethon objects ? pure strings in, CommandResult out. The Telethon
5     adapter (commands.py) is tested separately via test_commands.py.
6     """
7
8     from __future__ import annotations
9
10    import aiosqlite
11    import pytest
12
13    from tg_video_filter import db
14    from tg_video_filter.command_service import (
15        CommandService,
16        format_commands,
17        parse_duration_s,
18        parse_size_mb,
19    )
20    from tg_video_filter.models import FilterConfig
21
22    USER_ID = 111
23
24
25    @pytest.fixture
26    async def conn() -> aiosqlite.Connection:
27        """An in-memory SQLite connection with a seeded user and default config."""
28        c = await aiosqlite.connect(":memory:")
29        await db._apply_schema(c)
30        await db.upsert_user(
31            c,
32            db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33        )
34        await db.save_filter_config(c, USER_ID, FilterConfig())
35        yield c
36        await c.close()
37
38
39    @pytest.fixture
40    def svc() -> CommandService:
41        return CommandService()
42
43
44    # --- unit parser tests -----
45
46
47    @pytest.mark.parametrize(
48        "value, unit, expected",
49        [
50            ("10", "MB", 10),
51            ("1", "GB", 1024),

```

```

52         ("500", "KB", 1),
53         ("5", "", 5),
54         ("2", "TB", 2 * 1024 * 1024),
55     ],
56 )
57 def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
58     assert parse_size_mb(value, unit) == expected
59
60
61 def test_parse_size_mb_unknown_unit_raises() -> None:
62     with pytest.raises(ValueError):
63         parse_size_mb("10", "XB")
64
65
66 @pytest.mark.parametrize(
67     "value, unit, expected",
68     [
69         ("15", "m", 900),
70         ("2", "h", 7200),
71         ("90", "s", 90),
72         ("5", "", 5),
73     ],
74 )
75 def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
76     assert parse_duration_s(value, unit) == expected
77
78
79 def test_parse_duration_s_unknown_unit_raises() -> None:
80     with pytest.raises(ValueError):
81         parse_duration_s("10", "x")
82
83
84 # --- /set size -----
85
86
87 async def test_set_size(conn: aiomysqlite.Connection, svc: CommandService) -> None:
88     result = await svc.handle(USER_ID, "/set size 100MB", conn, FilterConfig())
89     assert "100MB" in result.reply_text
90     assert result.config_changed is True
91
92
93 async def test_set_size_invalid_format_returns_empty(
94     conn: aiomysqlite.Connection, svc: CommandService
95 ) -> None:
96     """Unparseable input that doesn't match the regex returns empty result (silent)."""
97     result = await svc.handle(USER_ID, "/set size not-a-number", conn, FilterConfig())
98     assert result.reply_text == "" # regex doesn't match, no reply
99     assert result.config_changed is False
100
101
102 async def test_set_size_below_minimum(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
103     result = await svc.handle(USER_ID, "/set size 0MB", conn, FilterConfig())
104     assert "at least 1 MB" in result.reply_text
105     assert result.config_changed is False
106
107
108 # --- /set length -----
109
110
111 async def test_set_length(conn: aiomysqlite.Connection, svc: CommandService) -> None:
112     result = await svc.handle(USER_ID, "/set length 15m", conn, FilterConfig())
113     assert "15m" in result.reply_text or "900s" in result.reply_text
114     assert result.config_changed is True
115

```

```

116
117 async def test_set_length_below_min(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
118     cfg = FilterConfig(min_duration_s=60)
119     result = await svc.handle(USER_ID, "/set length 30s", conn, cfg)
120     assert "must be >= min duration" in result.reply_text
121     assert result.config_changed is False
122
123
124 # --- /set min length -----
125
126
127 async def test_set_min_length(conn: aiomysqlite.Connection, svc: CommandService) -> None:
128     result = await svc.handle(USER_ID, "/set min length 5s", conn, FilterConfig())
129     assert "5s" in result.reply_text
130     assert result.config_changed is True
131
132
133 async def test_set_min_length_above_max(conn: aiomysqlite.Connection, svc: CommandService)
-> None:
134     cfg = FilterConfig(max_duration_s=10)
135     result = await svc.handle(USER_ID, "/set min length 60s", conn, cfg)
136     assert "must be <= max duration" in result.reply_text
137     assert result.config_changed is False
138
139
140 # --- /set min resolution -----
141
142
143 async def test_set_min_resolution(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
144     result = await svc.handle(USER_ID, "/set min resolution 720p", conn, FilterConfig())
145     assert "720p" in result.reply_text
146     assert result.config_changed is True
147
148
149 async def test_set_min_resolution_below_min(
150     conn: aiomysqlite.Connection, svc: CommandService
151 ) -> None:
152     result = await svc.handle(USER_ID, "/set min resolution 0p", conn, FilterConfig())
153     assert "at least 1p" in result.reply_text
154     assert result.config_changed is False
155
156
157 async def test_set_min_res_help_on_malformed(
158     conn: aiomysqlite.Connection, svc: CommandService
159 ) -> None:
160     result = await svc.handle(USER_ID, "/set min resolution abc", conn, FilterConfig())
161     assert "Usage:" in result.reply_text
162
163
164 # --- /set exclude -----
165
166
167 async def test_set_exclude_gif_on(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
168     result = await svc.handle(USER_ID, "/set exclude gif on", conn, FilterConfig())
169     assert "ON" in result.reply_text
170     assert result.config_changed is True
171
172
173 async def test_set_exclude_round_off(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
174     result = await svc.handle(USER_ID, "/set exclude round off", conn, FilterConfig())
175     assert "OFF" in result.reply_text

```

```

176         assert result.config_changed is True
177
178
179     async def test_set_exclude_help_on_malformed(
180         conn: aiomysqlite.Connection, svc: CommandService
181     ) -> None:
182         result = await svc.handle(USER_ID, "/set exclude gif true", conn, FilterConfig())
183         assert "Usage:" in result.reply_text
184         assert result.config_changed is False
185
186
187     # --- /show config -----
188
189
190     async def test_show_config(conn: aiomysqlite.Connection, svc: CommandService) -> None:
191         result = await svc.handle(USER_ID, "/show config", conn, FilterConfig())
192         assert "Current Filter Configuration" in result.reply_text
193         assert result.config_changed is False
194
195
196     # --- /show commands -----
197
198
199     async def test_show_commands(conn: aiomysqlite.Connection, svc: CommandService) -> None:
200         result = await svc.handle(USER_ID, "/show commands", conn, FilterConfig())
201         assert "Available Commands" in result.reply_text
202         assert result.config_changed is False
203
204
205     # --- unrecognized command -----
206
207
208     async def test_unrecognized_returns_empty(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
209         result = await svc.handle(USER_ID, "hello world", conn, FilterConfig())
210         assert result.reply_text == ""
211         assert result.config_changed is False
212
213
214     # --- format_commands -----
215
216
217     def test_format_commands_includes_all_sections() -> None:
218         text = format_commands()
219         assert "/show config" in text
220         assert "/set size" in text
221         assert "/set exclude" in text
222         assert "/backfill" in text
223

```

7. assistant (2026-07-06T08:39:37.743Z)

I have the key files. Now let me verify behavioral equivalence with a battery test through both old and new code paths, and check specific divergence points.

8. user (2026-07-06T08:39:40.300Z)

```

1     """Tests for the commands module (config commands: /set size, /set length, /show
config).
2
3     Pure functions (parsing, formatting, regex) are tested without Telethon.
4     DB-persistence tests use an in-memory SQLite database.
5     The async handler is tested with mocked Telethon event/client objects.
6     """
7

```

```

8     from __future__ import annotations
9
10    from unittest.mock import AsyncMock, MagicMock
11
12    import aiosqlite
13    import pytest
14    from telethon.errors import RPCError
15
16    from tg_video_filter import db
17    from tg_video_filter.commands import (
18        _EXCLUDE_RE,
19        _LENGTH_RE,
20        _MIN_LENGTH_RE,
21        _MIN_RES_HELP_RE,
22        _MIN_RES_RE,
23        _SHOW_COMMANDS_RE,
24        _SHOW_CONFIG_RE,
25        _SIZE_RE,
26        ENGINE_KEY,
27        _format_duration,
28        _format_size,
29        _update_config,
30        format_commands,
31        format_config,
32        make_config_command_handler,
33        parse_duration_s,
34        parse_size_mb,
35    )
36    from tg_video_filter.filters import FilterEngine
37    from tg_video_filter.models import DeliveryMode, FilterConfig, User
38
39    # --- parse_size_mb -----
40
41
42    @pytest.mark.parametrize(
43        ("value", "unit", "expected"),
44        [
45            ("10", "MB", 10),
46            ("10", "mb", 10),
47            ("10", "M", 10),
48            ("10", "m", 10),
49            ("10", "", 10),
50            ("1", "GB", 1024),
51            ("1", "gb", 1024),
52            ("1", "G", 1024),
53            ("1", "g", 1024),
54            ("500", "KB", 1), # 500 KiB ? 0.49 MiB, floored to 1
55            ("1024", "KB", 1),
56            ("2048", "KB", 2),
57            ("0", "MB", 0), # parser returns 0; handler rejects (tested below)
58            ("1", "TB", 1024 * 1024),
59        ],
60    )
61    def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
62        assert parse_size_mb(value, unit) == expected
63
64
65    def test_parse_size_mb_unknown_unit_raises() -> None:
66        with pytest.raises(ValueError, match="Unknown size unit"):
67            parse_size_mb("10", "XB")
68
69
70    # --- parse_duration_s -----
71
72

```

```

73     @pytest.mark.parametrize(
74         ("value", "unit", "expected"),
75         [
76             ("90", "s", 90),
77             ("90", "S", 90),
78             ("90", "", 90),
79             ("15", "m", 900),
80             ("15", "M", 900),
81             ("2", "h", 7200),
82             ("2", "H", 7200),
83             ("0", "m", 0),
84             ("1", "h", 3600),
85         ],
86     )
87     def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
88         assert parse_duration_s(value, unit) == expected
89
90
91     def test_parse_duration_s_unknown_unit_raises() -> None:
92         with pytest.raises(ValueError, match="Unknown time unit"):
93             parse_duration_s("10", "d")
94
95
96     # --- _format_duration -----
97
98
99     @pytest.mark.parametrize(
100         ("seconds", "expected"),
101         [
102             (None, "unlimited"),
103             (0, "0s"),
104             (30, "30s"),
105             (90, "1m30s"),
106             (120, "2m"),
107             (3600, "1h"),
108             (3661, "1h1m1s"),
109             (7200, "2h"),
110             (7320, "2h2m"),
111         ],
112     )
113     def test_format_duration(seconds: int | None, expected: str) -> None:
114         assert _format_duration(seconds) == expected
115
116
117     # --- _format_size -----
118
119
120     @pytest.mark.parametrize(
121         ("mb", "expected"),
122         [
123             (None, "unlimited"),
124             (0, "0MB"),
125             (10, "10MB"),
126             (512, "512MB"),
127             (1024, "1GB"),
128             (1536, "1.5GB"),
129             (2048, "2GB"),
130         ],
131     )
132     def test_format_size(mb: int | None, expected: str) -> None:
133         assert _format_size(mb) == expected
134
135
136     # --- format_config -----
137

```

```

138
139 def test_format_config_defaults() -> None:
140     cfg = FilterConfig()
141     result = format_config(cfg)
142     assert "???" in result
143     assert "5s" in result
144     assert "2h" in result
145     assert "1080p" in result
146     assert "2GB" in result
147     assert "forward" in result
148     assert "me" in result
149
150
151 def test_format_config_custom() -> None:
152     cfg = FilterConfig(
153         min_duration_s=10,
154         max_duration_s=600,
155         target_height=720,
156         max_size_mb=512,
157         delivery_mode=DeliveryMode.BOTH,
158         delivery_target="@mychannel",
159     )
160     result = format_config(cfg)
161     assert "10s" in result
162     assert "10m" in result
163     assert "720p" in result
164     assert "512MB" in result
165     assert "both" in result
166     assert "@mychannel" in result
167
168
169 def test_format_config_disabled_resolution() -> None:
170     cfg = FilterConfig(target_height=None)
171     result = format_config(cfg)
172     assert "disabled" in result
173
174
175 # --- regex: /set size -----
176
177
178 @pytest.mark.parametrize(
179     ("text", "expected_value", "expected_unit"),
180     [
181         ("/set size 10MB", "10", "MB"),
182         ("/set size 10mb", "10", "mb"),
183         ("/set size 10M", "10", "M"),
184         ("/set size 1GB", "1", "GB"),
185         ("/set size 500KB", "500", "KB"),
186         ("/set size 10", "10", ""),
187         ("/set size 0MB", "0", "MB"),
188         ("/set size 20MB", "20", "MB"), # extra whitespace
189         ("/SET SIZE 10MB", "10", "MB"), # case-insensitive
190     ],
191 )
192 def test_size_regex_matches(text: str, expected_value: str, expected_unit: str) -> None:
193     m = _SIZE_RE.match(text)
194     assert m is not None
195     assert m.group(1) == expected_value
196     assert m.group(2) == expected_unit
197
198
199 @pytest.mark.parametrize(
200     "text",
201     [
202         "/set size",

```



```

203         "/set size abc",
204         "/set size -1MB",
205         "/set size 10.5MB",
206         "/set size MB",
207         "/setsize 10MB",
208         "/set size 10XB",
209         "random message",
210         "",
211     ],
212 )
213 def test_size_regex_no_match(text: str) -> None:
214     assert _SIZE_RE.match(text) is None
215
216
217 # --- regex: /set length -----
218
219
220 @pytest.mark.parametrize(
221     ("text", "expected_value", "expected_unit"),
222     [
223         ("/set length 15m", "15", "m"),
224         ("/set length 15M", "15", "M"),
225         ("/set length 90s", "90", "s"),
226         ("/set length 2h", "2", "h"),
227         ("/set length 90", "90", ""),
228         ("/set length 30m", "30", "m"), # extra whitespace
229         ("/SET LENGTH 15m", "15", "m"), # case-insensitive
230     ],
231 )
232 def test_length_regex_matches(text: str, expected_value: str, expected_unit: str) ->
None:
233     m = _LENGTH_RE.match(text)
234     assert m is not None
235     assert m.group(1) == expected_value
236     assert m.group(2) == expected_unit
237
238
239 @pytest.mark.parametrize(
240     "text",
241     [
242         "/set length",
243         "/set length abc",
244         "/set length -1m",
245         "/set length 10.5m",
246         "/set length m",
247         "/setlength 15m",
248         "/set length 10d",
249         "random message",
250         "",
251     ],
252 )
253 def test_length_regex_no_match(text: str) -> None:
254     assert _LENGTH_RE.match(text) is None
255
256
257 # --- regex: /show config -----
258
259
260 @pytest.mark.parametrize(
261     "text",
262     [
263         "/show config",
264         "/SHOW CONFIG",
265         "/Show Config",
266     ],

```

```

267 )
268 def test_show_config_regex_matches(text: str) -> None:
269     assert _SHOW_CONFIG_RE.match(text) is not None
270
271
272 @pytest.mark.parametrize(
273     "text",
274     [
275         "/show",
276         "/show config ",
277         "/show config extra",
278         "/showconfig",
279         "show config",
280         "",
281     ],
282 )
283 def test_show_config_regex_no_match(text: str) -> None:
284     assert _SHOW_CONFIG_RE.match(text) is None
285
286
287 # --- regex: /show commands -----
288
289
290 @pytest.mark.parametrize(
291     "text",
292     [
293         "/show commands",
294         "/SHOW COMMANDS",
295         "/Show Commands",
296     ],
297 )
298 def test_show_commands_regex_matches(text: str) -> None:
299     assert _SHOW_COMMANDS_RE.match(text) is not None
300
301
302 @pytest.mark.parametrize(
303     "text",
304     [
305         "/show",
306         "/show commands ",
307         "/show commands extra",
308         "/showcommands",
309         "show commands",
310         "/show config",
311         "",
312     ],
313 )
314 def test_show_commands_regex_no_match(text: str) -> None:
315     assert _SHOW_COMMANDS_RE.match(text) is None
316
317
318 # --- format_commands -----
319
320
321 def test_format_commands() -> None:
322     result = format_commands()
323     assert "/show config" in result
324     assert "/set size" in result
325     assert "/set length" in result
326     assert "/backfill" in result
327     assert "/stop" in result
328     assert "/show commands" in result
329
330
331 # --- DB persistence -----

```

```

332
333
334 @pytest.fixture
335 async def memory_conn() -> aiosqlite.Connection:
336     """In-memory DB with schema applied."""
337     async with aiosqlite.connect(":memory:") as conn:
338         await db._apply_schema(conn)
339         # Seed a user so FK constraints are satisfied.
340         await db.upsert_user(
341             conn, __import__("tg_video_filter.models", fromlist=["User"]).User(id=1)
342         )
343     yield conn
344
345
346 async def test_load_default_config(memory_conn: aiosqlite.Connection) -> None:
347     """A user with no saved config gets defaults."""
348     cfg = await db.load_filter_config(memory_conn, 1)
349     assert cfg.max_size_mb == 2048
350     assert cfg.max_duration_s == 7200
351     assert cfg.min_duration_s == 5
352
353
354 async def test_save_and_load_config(memory_conn: aiosqlite.Connection) -> None:
355     """Save a config and load it back ? round-trip."""
356     cfg = FilterConfig(max_size_mb=512, max_duration_s=600)
357     await db.save_filter_config(memory_conn, 1, cfg)
358     loaded = await db.load_filter_config(memory_conn, 1)
359     assert loaded.max_size_mb == 512
360     assert loaded.max_duration_s == 600
361     # Unchanged fields keep defaults.
362     assert loaded.min_duration_s == 5
363
364
365 async def test_config_persistence_survives_reload(memory_conn: aiosqlite.Connection) ->
None:
366     """Config written once is readable after closing and reopening."""
367     cfg = FilterConfig(max_size_mb=256)
368     await db.save_filter_config(memory_conn, 1, cfg)
369
370     # Reload within the same connection (simulates restart).
371     loaded = await db.load_filter_config(memory_conn, 1)
372     assert loaded.max_size_mb == 256
373
374
375 # --- _update_config -----
376
377 USER_ID = 1
378
379
380 @pytest.fixture
381 def user() -> User:
382     return User(id=USER_ID)
383
384
385 @pytest.fixture
386 def engine_holder() -> dict[str, FilterEngine]:
387     return {ENGINE_KEY: FilterEngine(FilterConfig())}
388
389
390 def _make_mock_event(text: str, *, out: bool = True, peer_user_id: int = USER_ID) ->
MagicMock:
391     """Build a MagicMock that looks like a Telethon NewMessage event."""
392     event = MagicMock()
393     event.message.out = out
394     event.message.peer_id.user_id = peer_user_id

```

```

395         event.message.message = text
396         event.client.send_message = AsyncMock()
397         return event
398
399
400     async def test_update_config_sets_max_size(memory_conn: aioredis.Connection) -> None:
401         cfg = await _update_config(memory_conn, USER_ID, max_size_mb=512)
402         assert cfg.max_size_mb == 512
403         # Other fields unchanged.
404         assert cfg.max_duration_s == 7200
405         # Persisted.
406         loaded = await db.load_filter_config(memory_conn, USER_ID)
407         assert loaded.max_size_mb == 512
408
409
410     async def test_update_config_sets_max_duration(memory_conn: aioredis.Connection) ->
None:
411         cfg = await _update_config(memory_conn, USER_ID, max_duration_s=600)
412         assert cfg.max_duration_s == 600
413         assert cfg.max_size_mb == 2048 # unchanged
414
415
416     async def test_update_config_sets_both_fields(memory_conn: aioredis.Connection) ->
None:
417         cfg = await _update_config(memory_conn, USER_ID, max_size_mb=256,
max_duration_s=300)
418         assert cfg.max_size_mb == 256
419         assert cfg.max_duration_s == 300
420
421
422     async def test_update_config_no_fields_is_noop(memory_conn: aioredis.Connection) ->
None:
423         cfg = await _update_config(memory_conn, USER_ID)
424         assert cfg.max_size_mb == 2048 # default
425         assert cfg.max_duration_s == 7200 # default
426
427
428     async def test_update_config_preserves_delivery_mode(memory_conn: aioredis.Connection)
-> None:
429         # Save a custom delivery mode first.
430         base = FilterConfig(delivery_mode=DeliveryMode.BOTH)
431         await db.save_filter_config(memory_conn, USER_ID, base)
432         cfg = await _update_config(memory_conn, USER_ID, max_size_mb=100)
433         assert cfg.delivery_mode == DeliveryMode.BOTH
434         assert cfg.max_size_mb == 100
435
436
437     # --- handler: message filtering (non-command messages) -----
438
439
440     async def test_handler_ignores_non_outgoing(
441         memory_conn: aioredis.Connection, user: User, engine_holder: dict
442     ) -> None:
443         handler = make_config_command_handler(user, memory_conn, engine_holder)
444         event = _make_mock_event("/show config", out=False)
445         await handler(event)
446         event.client.send_message.assert_not_called()
447
448
449     async def test_handler_ignores_wrong_user(
450         memory_conn: aioredis.Connection, user: User, engine_holder: dict
451     ) -> None:
452         handler = make_config_command_handler(user, memory_conn, engine_holder)
453         event = _make_mock_event("/show config", peer_user_id=999)
454         await handler(event)

```

```

455         event.client.send_message.assert_not_called()
456
457
458     async def test_handler_ignores_missing_message(
459         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
460     ) -> None:
461         handler = make_config_command_handler(user, memory_conn, engine_holder)
462         event = MagicMock()
463         event.message = None
464         await handler(event) # must not crash
465
466
467     async def test_handler_ignores_unknown_command(
468         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
469     ) -> None:
470         handler = make_config_command_handler(user, memory_conn, engine_holder)
471         event = _make_mock_event("hello world")
472         await handler(event)
473         event.client.send_message.assert_not_called()
474
475
476     # --- handler: /show config -----
477
478
479     async def test_handler_show_config(
480         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
481     ) -> None:
482         handler = make_config_command_handler(user, memory_conn, engine_holder)
483         event = _make_mock_event("/show config")
484         await handler(event)
485         event.client.send_message.assert_called_once()
486         args, _kwargs = event.client.send_message.call_args
487         assert args[0] == "me"
488         assert "Current Filter Configuration" in args[1]
489
490
491     async def test_handler_show_config_reflects_saved_config(
492         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
493     ) -> None:
494         # Save a custom config first.
495         cfg = FilterConfig(max_size_mb=100, max_duration_s=300)
496         await db.save_filter_config(memory_conn, USER_ID, cfg)
497
498         handler = make_config_command_handler(user, memory_conn, engine_holder)
499         event = _make_mock_event("/show config")
500         await handler(event)
501         args, _kwargs = event.client.send_message.call_args
502         assert "100MB" in args[1]
503         assert "5m" in args[1]
504
505
506     # --- handler: /show commands -----
507
508
509     async def test_handler_show_commands(
510         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
511     ) -> None:
512         handler = make_config_command_handler(user, memory_conn, engine_holder)
513         event = _make_mock_event("/show commands")
514         await handler(event)
515         event.client.send_message.assert_called_once()
516         args, _kwargs = event.client.send_message.call_args
517         assert args[0] == "me"
518         assert "/set size" in args[1]
519         assert "/set length" in args[1]

```

```

520     assert "/backfill" in args[1]
521     assert "/show config" in args[1]
522
523
524 # --- handler: /set size -----
525
526
527 async def test_handler_set_size_persists_and_replies(
528     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
529 ) -> None:
530     handler = make_config_command_handler(user, memory_conn, engine_holder)
531     event = _make_mock_event("/set size 512MB")
532     await handler(event)
533
534     # Reply sent.
535     event.client.send_message.assert_called_once()
536     args, _kwargs = event.client.send_message.call_args
537     assert "512MB" in args[1]
538     assert "?" in args[1]
539
540     # Persisted to DB.
541     loaded = await db.load_filter_config(memory_conn, USER_ID)
542     assert loaded.max_size_mb == 512
543
544     # Engine hot-reloaded.
545     assert engine_holder[ENGINE_KEY].cfg.max_size_mb == 512
546
547
548 async def test_handler_set_size_gb(
549     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
550 ) -> None:
551     handler = make_config_command_handler(user, memory_conn, engine_holder)
552     event = _make_mock_event("/set size 1GB")
553     await handler(event)
554
555     loaded = await db.load_filter_config(memory_conn, USER_ID)
556     assert loaded.max_size_mb == 1024
557     assert engine_holder[ENGINE_KEY].cfg.max_size_mb == 1024
558
559
560 async def test_handler_set_size_invalid_unit_silently_ignored(
561     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
562 ) -> None:
563     """Units not matching the regex (e.g. XB) are silently ignored, not replied to."""
564     handler = make_config_command_handler(user, memory_conn, engine_holder)
565     event = _make_mock_event("/set size 10XB")
566     await handler(event)
567
568     # Regex doesn't match, so no reply is sent.
569     event.client.send_message.assert_not_called()
570
571     # Config unchanged.
572     loaded = await db.load_filter_config(memory_conn, USER_ID)
573     assert loaded.max_size_mb == 2048 # still default
574
575
576 async def test_handler_set_size_engine_reloaded_for_subsequent_messages(
577     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
578 ) -> None:
579     handler = make_config_command_handler(user, memory_conn, engine_holder)
580
581     # First: set size.
582     e1 = _make_mock_event("/set size 100MB")
583     await handler(e1)
584     assert engine_holder[ENGINE_KEY].cfg.max_size_mb == 100

```

```

585
586     # Then: show config ? should reflect the new size.
587     e2 = _make_mock_event("/show config")
588     await handler(e2)
589     args, _kwargs = e2.client.send_message.call_args
590     assert "100MB" in args[1]
591
592
593     # --- handler: /set length -----
594
595
596     async def test_handler_set_length_persists_and_replies(
597         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
598     ) -> None:
599         handler = make_config_command_handler(user, memory_conn, engine_holder)
600         event = _make_mock_event("/set length 10m")
601         await handler(event)
602
603         event.client.send_message.assert_called_once()
604         args, _kwargs = event.client.send_message.call_args
605         assert "10m" in args[1]
606         assert "?" in args[1]
607
608         loaded = await db.load_filter_config(memory_conn, USER_ID)
609         assert loaded.max_duration_s == 600 # 10 * 60
610         assert engine_holder[ENGINE_KEY].cfg.max_duration_s == 600
611
612
613     async def test_handler_set_length_hours(
614         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
615     ) -> None:
616         handler = make_config_command_handler(user, memory_conn, engine_holder)
617         event = _make_mock_event("/set length 2h")
618         await handler(event)
619
620         loaded = await db.load_filter_config(memory_conn, USER_ID)
621         assert loaded.max_duration_s == 7200
622         assert engine_holder[ENGINE_KEY].cfg.max_duration_s == 7200
623
624
625     async def test_handler_set_length_invalid_unit_silently_ignored(
626         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
627     ) -> None:
628         """Units not matching the regex (e.g. d) are silently ignored, not replied to."""
629         handler = make_config_command_handler(user, memory_conn, engine_holder)
630         event = _make_mock_event("/set length 10d")
631         await handler(event)
632
633         # Regex doesn't match, so no reply is sent.
634         event.client.send_message.assert_not_called()
635
636         loaded = await db.load_filter_config(memory_conn, USER_ID)
637         assert loaded.max_duration_s == 7200 # still default
638
639
640     async def test_handler_set_length_seconds(
641         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
642     ) -> None:
643         handler = make_config_command_handler(user, memory_conn, engine_holder)
644         event = _make_mock_event("/set length 90s")
645         await handler(event)
646
647         loaded = await db.load_filter_config(memory_conn, USER_ID)
648         assert loaded.max_duration_s == 90
649

```

```

650
651 # --- handler: _reply error resilience -----
652
653
654 async def test_handler_reply_error_is_suppressed(
655     memory_conn: aiomysqlite.Connection, user: User, engine_holder: dict
656 ) -> None:
657     handler = make_config_command_handler(user, memory_conn, engine_holder)
658     event = _make_mock_event("/show config")
659     event.client.send_message.side_effect = OSError("network down")
660
661     # Must not raise ? OSError is in the caught tuple.
662     await handler(event)
663
664
665 async def test_handler_reply_rpcerror_is_suppressed(
666     memory_conn: aiomysqlite.Connection, user: User, engine_holder: dict
667 ) -> None:
668     handler = make_config_command_handler(user, memory_conn, engine_holder)
669     event = _make_mock_event("/show config")
670     event.client.send_message.side_effect = RPCError(400, "CHAT_WRITE_FORBIDDEN")
671
672     # Must not raise ? RPCError is in the caught tuple.
673     await handler(event)
674
675
676 async def test_handler_reply_typeerror_is_not_suppressed(
677     memory_conn: aiomysqlite.Connection, user: User, engine_holder: dict
678 ) -> None:
679     """Programming errors (TypeError, etc.) still surface ? only network/RPC errors are
680 caught."""
681     handler = make_config_command_handler(user, memory_conn, engine_holder)
682     event = _make_mock_event("/show config")
683     event.client.send_message.side_effect = TypeError("bad arg")
684
685     with pytest.raises(TypeError):
686         await handler(event)
687
688
689 async def test_handler_set_size_reply_error_still_persists(
690     memory_conn: aiomysqlite.Connection, user: User, engine_holder: dict
691 ) -> None:
692     handler = make_config_command_handler(user, memory_conn, engine_holder)
693     event = _make_mock_event("/set size 256MB")
694     event.client.send_message.side_effect = ConnectionError("network down")
695
696     # Must not raise.
697     await handler(event)
698
699     # Config still persisted despite reply failure.
700     loaded = await db.load_filter_config(memory_conn, USER_ID)
701     assert loaded.max_size_mb == 256
702
703 # --- handler: case insensitivity -----
704
705
706 async def test_handler_case_insensitive_set_size(
707     memory_conn: aiomysqlite.Connection, user: User, engine_holder: dict
708 ) -> None:
709     handler = make_config_command_handler(user, memory_conn, engine_holder)
710     event = _make_mock_event("/SET SIZE 50mb")
711     await handler(event)
712
713     loaded = await db.load_filter_config(memory_conn, USER_ID)

```



```

714         assert loaded.max_size_mb == 50
715
716
717     async def test_handler_case_insensitive_show_commands(
718         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
719     ) -> None:
720         handler = make_config_command_handler(user, memory_conn, engine_holder)
721         event = _make_mock_event("/SHOW COMMANDS")
722         await handler(event)
723
724         event.client.send_message.assert_called_once()
725         args, _kwargs = event.client.send_message.call_args
726         assert "Available Commands" in args[1]
727
728
729     # --- handler: validation (size >= 1) -----
730
731
732     async def test_handler_set_size_zero_rejected(
733         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
734     ) -> None:
735         """Setting size to 0 writes an invalid config (< ge=1) ? must be rejected."""
736         handler = make_config_command_handler(user, memory_conn, engine_holder)
737         event = _make_mock_event("/set size 0MB")
738         await handler(event)
739
740         event.client.send_message.assert_called_once()
741         args, _kwargs = event.client.send_message.call_args
742         assert "at least 1 MB" in args[1]
743
744         # Config not changed.
745         loaded = await db.load_filter_config(memory_conn, USER_ID)
746         assert loaded.max_size_mb == 2048 # still default
747
748
749     async def test_handler_set_size_one_accepted(
750         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
751     ) -> None:
752         handler = make_config_command_handler(user, memory_conn, engine_holder)
753         event = _make_mock_event("/set size 1MB")
754         await handler(event)
755
756         loaded = await db.load_filter_config(memory_conn, USER_ID)
757         assert loaded.max_size_mb == 1
758
759
760     # --- handler: validation (max_duration >= min_duration) -----
761
762
763     async def test_handler_set_length_below_min_rejected(
764         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
765     ) -> None:
766         """Setting max_duration < min_duration (default 5s) must be rejected."""
767         handler = make_config_command_handler(user, memory_conn, engine_holder)
768         event = _make_mock_event("/set length 2s")
769         await handler(event)
770
771         event.client.send_message.assert_called_once()
772         args, _kwargs = event.client.send_message.call_args
773         assert "must be >= min duration" in args[1]
774
775         loaded = await db.load_filter_config(memory_conn, USER_ID)
776         assert loaded.max_duration_s == 7200 # still default
777
778

```

```

779     async def test_handler_set_length_equal_to_min_accepted(
780         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
781     ) -> None:
782         """Setting max_duration == min_duration (5s) is valid."""
783         handler = make_config_command_handler(user, memory_conn, engine_holder)
784         event = _make_mock_event("/set length 5s")
785         await handler(event)
786
787         loaded = await db.load_filter_config(memory_conn, USER_ID)
788         assert loaded.max_duration_s == 5
789
790
791     async def test_handler_set_length_below_custom_min_rejected(
792         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
793     ) -> None:
794         """When user has a custom min_duration_s, the check respects it."""
795         # Save a config with min_duration_s=60.
796         cfg = FilterConfig(min_duration_s=60)
797         await db.save_filter_config(memory_conn, USER_ID, cfg)
798
799         handler = make_config_command_handler(user, memory_conn, engine_holder)
800         event = _make_mock_event("/set length 30s")
801         await handler(event)
802
803         args, _kwargs = event.client.send_message.call_args
804         assert ">= min duration" in args[1]
805         assert "1m" in args[1] # min_duration formatted
806
807         loaded = await db.load_filter_config(memory_conn, USER_ID)
808         assert loaded.max_duration_s == 7200 # unchanged (wasn't persisted because
rejected)
809
810
811     # --- handler: trailing whitespace -----
812
813
814     async def test_handler_trailing_whitespace_accepted(
815         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
816     ) -> None:
817         """Trailing whitespace (common on mobile keyboards) is stripped before matching."""
818         handler = make_config_command_handler(user, memory_conn, engine_holder)
819
820         for text in ("/show config ", "/show config  ", "/show commands ", "/set size 10MB
"):
821             event = _make_mock_event(text)
822             await handler(event)
823             event.client.send_message.assert_called_once()
824             # Reset mock for next iteration.
825             event.client.send_message.reset_mock()
826
827
828     async def test_handler_leading_whitespace_not_matched(
829         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
830     ) -> None:
831         """Only trailing whitespace is stripped; leading whitespace is unusual and not
handled."""
832         handler = make_config_command_handler(user, memory_conn, engine_holder)
833         event = _make_mock_event(" /show config")
834         await handler(event)
835         event.client.send_message.assert_not_called()
836
837
838     # --- regex: /set max length (alias for /set length) -----
839
840

```

```

841 @pytest.mark.parametrize(
842     ("text", "expected_value", "expected_unit"),
843     [
844         ("/set max length 15m", "15", "m"),
845         ("/set max length 2h", "2", "h"),
846         ("/set max length 90s", "90", "s"),
847         ("/set max length 90", "90", ""),
848         ("/SET MAX LENGTH 15m", "15", "m"), # case-insensitive
849     ],
850 )
851 def test_max_length_regex_matches(text: str, expected_value: str, expected_unit: str) ->
None:
852     m = _LENGTH_RE.match(text)
853     assert m is not None
854     assert m.group(1) == expected_value
855     assert (m.group(2) or "") == expected_unit
856
857
858 # --- regex: /set min length -----
859
860
861 @pytest.mark.parametrize(
862     ("text", "expected_value", "expected_unit"),
863     [
864         ("/set min length 5s", "5", "s"),
865         ("/set min length 1m", "1", "m"),
866         ("/set min length 30", "30", ""),
867         ("/set min length 10s", "10", "s"), # extra whitespace
868         ("/SET MIN LENGTH 5s", "5", "s"), # case-insensitive
869     ],
870 )
871 def test_min_length_regex_matches(text: str, expected_value: str, expected_unit: str) ->
None:
872     m = _MIN_LENGTH_RE.match(text)
873     assert m is not None
874     assert m.group(1) == expected_value
875     assert (m.group(2) or "") == expected_unit
876
877
878 @pytest.mark.parametrize(
879     "text",
880     [
881         "/set min length",
882         "/set min length abc",
883         "/set min length -1s",
884         "/set minlength 5s",
885         "",
886     ],
887 )
888 def test_min_length_regex_no_match(text: str) -> None:
889     assert _MIN_LENGTH_RE.match(text) is None
890
891
892 # --- handler: /set min length -----
893
894
895 async def test_handler_set_min_length_persists_and_replies(
896     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
897 ) -> None:
898     handler = make_config_command_handler(user, memory_conn, engine_holder)
899     event = _make_mock_event("/set min length 10s")
900     await handler(event)
901
902     event.client.send_message.assert_called_once()
903     args, _kwargs = event.client.send_message.call_args

```

```

904         assert "10s" in args[1]
905         assert "?" in args[1]
906
907         loaded = await db.load_filter_config(memory_conn, USER_ID)
908         assert loaded.min_duration_s == 10
909         assert engine_holder[ENGINE_KEY].cfg.min_duration_s == 10
910
911
912     async def test_handler_set_min_length_minutes(
913         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
914     ) -> None:
915         handler = make_config_command_handler(user, memory_conn, engine_holder)
916         event = _make_mock_event("/set min length 2m")
917         await handler(event)
918
919         loaded = await db.load_filter_config(memory_conn, USER_ID)
920         assert loaded.min_duration_s == 120
921
922
923     async def test_handler_set_min_length_exceeds_max_rejected(
924         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
925     ) -> None:
926         """Setting min_duration > max_duration must be rejected."""
927         # Save a config with max_duration_s=60.
928         cfg = FilterConfig(max_duration_s=60)
929         await db.save_filter_config(memory_conn, USER_ID, cfg)
930
931         handler = make_config_command_handler(user, memory_conn, engine_holder)
932         event = _make_mock_event("/set min length 120s")
933         await handler(event)
934
935         args, _kwargs = event.client.send_message.call_args
936         assert "<= max duration" in args[1]
937
938         loaded = await db.load_filter_config(memory_conn, USER_ID)
939         assert loaded.min_duration_s == 5 # unchanged (default)
940
941
942     async def test_handler_set_min_length_zero_accepted(
943         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
944     ) -> None:
945         """min_duration_s=0 is valid (FilterConfig allows ge=0)."""
946         handler = make_config_command_handler(user, memory_conn, engine_holder)
947         event = _make_mock_event("/set min length 0s")
948         await handler(event)
949
950         loaded = await db.load_filter_config(memory_conn, USER_ID)
951         assert loaded.min_duration_s == 0
952
953
954     # --- handler: /set max length (alias) -----
955
956
957     async def test_handler_set_max_length_same_as_set_length(
958         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
959     ) -> None:
960         """/set max length is an alias for /set length ? both set max_duration_s."""
961         handler = make_config_command_handler(user, memory_conn, engine_holder)
962         event = _make_mock_event("/set max length 5m")
963         await handler(event)
964
965         loaded = await db.load_filter_config(memory_conn, USER_ID)
966         assert loaded.max_duration_s == 300
967
968         args, _kwargs = event.client.send_message.call_args

```

```

969         assert "5m" in args[1]
970         assert "?" in args[1]
971
972
973 # --- regex: /set exclude -----
974
975
976 @pytest.mark.parametrize(
977     ("text", "expected_what", "expected_state"),
978     [
979         ("/set exclude gif on", "gif", "on"),
980         ("/set exclude gif off", "gif", "off"),
981         ("/set exclude round on", "round", "on"),
982         ("/set exclude round off", "round", "off"),
983         ("/set exclude nosound on", "nosound", "on"),
984         ("/set exclude nosound off", "nosound", "off"),
985         ("/SET EXCLUDE GIF ON", "GIF", "ON"), # case-insensitive, raw capture
986     ],
987 )
988 def test_exclude_regex_matches(text: str, expected_what: str, expected_state: str) ->
None:
989     m = _EXCLUDE_RE.match(text)
990     assert m is not None, f"expected match for {text!r}"
991     assert m.group(1) == expected_what
992     assert m.group(2) == expected_state
993
994
995 @pytest.mark.parametrize(
996     "text",
997     [
998         "/set exclude",
999         "/set exclude gif",
1000         "/set exclude gif yes",
1001         "/set exclude gif ON ",
1002         "/set exclude unknown on",
1003         "",
1004     ],
1005 )
1006 def test_exclude_regex_no_match(text: str) -> None:
1007     assert _EXCLUDE_RE.match(text) is None
1008
1009
1010 # --- handler: /set exclude -----
1011
1012
1013 async def test_handler_set_exclude_gif_on(
1014     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1015 ) -> None:
1016     handler = make_config_command_handler(user, memory_conn, engine_holder)
1017     event = _make_mock_event("/set exclude gif on")
1018     await handler(event)
1019
1020     event.client.send_message.assert_called_once()
1021     args, _kwargs = event.client.send_message.call_args
1022     assert "ON" in args[1]
1023     assert "GIF" in args[1]
1024
1025     loaded = await db.load_filter_config(memory_conn, USER_ID)
1026     assert loaded.exclude_gif is True
1027     assert engine_holder[ENGINE_KEY].cfg.exclude_gif is True
1028
1029
1030 async def test_handler_set_exclude_gif_off(
1031     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1032 ) -> None:

```

```

1033     # First enable it.
1034     cfg = FilterConfig(exclude_gif=True)
1035     await db.save_filter_config(memory_conn, USER_ID, cfg)
1036
1037     handler = make_config_command_handler(user, memory_conn, engine_holder)
1038     event = _make_mock_event("/set exclude gif off")
1039     await handler(event)
1040
1041     loaded = await db.load_filter_config(memory_conn, USER_ID)
1042     assert loaded.exclude_gif is False
1043
1044
1045     async def test_handler_set_exclude_round_on(
1046         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1047     ) -> None:
1048         handler = make_config_command_handler(user, memory_conn, engine_holder)
1049         event = _make_mock_event("/set exclude round on")
1050         await handler(event)
1051
1052         loaded = await db.load_filter_config(memory_conn, USER_ID)
1053         assert loaded.exclude_round is True
1054
1055
1056     async def test_handler_set_exclude_nosound_on(
1057         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1058     ) -> None:
1059         handler = make_config_command_handler(user, memory_conn, engine_holder)
1060         event = _make_mock_event("/set exclude nosound on")
1061         await handler(event)
1062
1063         loaded = await db.load_filter_config(memory_conn, USER_ID)
1064         assert loaded.exclude_nosound is True
1065
1066
1067     async def test_exclusions_defaults(
1068         memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1069     ) -> None:
1070         """GIFs and round videos are excluded by default; silent videos are not."""
1071         cfg = await db.load_filter_config(memory_conn, USER_ID)
1072         assert cfg.exclude_gif is True
1073         assert cfg.exclude_round is True
1074         assert cfg.exclude_nosound is False
1075
1076
1077     # --- format_config includes exclusions -----
1078
1079
1080     def test_format_config_shows_exclusions() -> None:
1081         cfg = FilterConfig()
1082         result = format_config(cfg)
1083         assert "GIF/animated" in result
1084         assert "Round videos" in result
1085         assert "Silent" in result
1086         assert "? allowed" in result # default = allowed
1087
1088
1089     def test_format_config_shows_exclusions_when_enabled() -> None:
1090         cfg = FilterConfig(exclude_gif=True, exclude_round=True, exclude_nosound=True)
1091         result = format_config(cfg)
1092         assert "? excluded" in result
1093         assert "? allowed" not in result
1094
1095
1096     # --- format_commands includes exclude section -----
1097

```

```

1098
1099 def test_format_commands_includes_exclude() -> None:
1100     result = format_commands()
1101     assert "/set exclude gif" in result
1102     assert "/set exclude round" in result
1103     assert "/set exclude nosound" in result
1104     assert "off by default" in result
1105
1106
1107 # --- regex: /set min resolution -----
1108
1109
1110 @pytest.mark.parametrize(
1111     ("text", "expected_height"),
1112     [
1113         ("/set min resolution 720p", 720),
1114         ("/set min resolution 1080p", 1080),
1115         ("/set min resolution 720", 720), # "p" is optional
1116         ("/set min res 480p", 480), # "res" shorthand
1117         ("/set min res 1080", 1080),
1118         ("/set min resolution 720p", 720), # extra whitespace
1119         ("/SET MIN RESOLUTION 720P", 720), # case-insensitive
1120     ],
1121 )
1122 def test_min_res_regex_matches(text: str, expected_height: int) -> None:
1123     m = _MIN_RES_RE.match(text)
1124     assert m is not None, f"expected match for {text!r}"
1125     assert int(m.group(1)) == expected_height
1126
1127
1128 @pytest.mark.parametrize(
1129     "text",
1130     [
1131         "/set min resolution",
1132         "/set min resolution abc",
1133         "/set min resolution -1p",
1134         "/set min res",
1135         "/set resolution 720p", # must have "min"
1136         "",
1137     ],
1138 )
1139 def test_min_res_regex_no_match(text: str) -> None:
1140     assert _MIN_RES_RE.match(text) is None
1141
1142
1143 # --- handler: /set min resolution -----
1144
1145
1146 async def test_handler_set_min_resolution_persists(
1147     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1148 ) -> None:
1149     handler = make_config_command_handler(user, memory_conn, engine_holder)
1150     event = _make_mock_event("/set min resolution 720p")
1151     await handler(event)
1152
1153     event.client.send_message.assert_called_once()
1154     args, _kwargs = event.client.send_message.call_args
1155     assert "720p" in args[1]
1156     assert "?" in args[1]
1157
1158     loaded = await db.load_filter_config(memory_conn, USER_ID)
1159     assert loaded.target_height == 720
1160     assert engine_holder[ENGINE_KEY].cfg.target_height == 720
1161
1162

```

```

1163 async def test_handler_set_min_resolution_zero_rejected(
1164     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1165 ) -> None:
1166     handler = make_config_command_handler(user, memory_conn, engine_holder)
1167     event = _make_mock_event("/set min resolution 0p")
1168     await handler(event)
1169
1170     args, _kwargs = event.client.send_message.call_args
1171     assert "at least 1p" in args[1]
1172
1173     loaded = await db.load_filter_config(memory_conn, USER_ID)
1174     assert loaded.target_height == 1080 # still default
1175
1176
1177 async def test_handler_set_min_resolution_shorthand(
1178     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1179 ) -> None:
1180     """/set min res 480p is accepted as shorthand.""
1181     handler = make_config_command_handler(user, memory_conn, engine_holder)
1182     event = _make_mock_event("/set min res 480p")
1183     await handler(event)
1184
1185     loaded = await db.load_filter_config(memory_conn, USER_ID)
1186     assert loaded.target_height == 480
1187
1188
1189 # --- format_commands includes min resolution -----
1190
1191
1192 def test_format_commands_includes_min_resolution() -> None:
1193     result = format_commands()
1194     assert "/set min resolution" in result
1195     assert "/set min res" in result
1196
1197
1198 # --- regex: /set min resolution help (catch-all) -----
1199
1200
1201 @pytest.mark.parametrize(
1202     "text",
1203     [
1204         "/set min resolution abc",
1205         "/set min resolution -1p",
1206         "/set min res xyz",
1207     ],
1208 )
1209 def test_min_res_help_regex_matches(text: str) -> None:
1210     assert _MIN_RES_HELP_RE.match(text) is not None
1211
1212
1213 @pytest.mark.parametrize(
1214     "text",
1215     [
1216         "/set min length 5s", # not resolution|res
1217         "", # empty
1218     ],
1219 )
1220 def test_min_res_help_regex_no_match(text: str) -> None:
1221     assert _MIN_RES_HELP_RE.match(text) is None
1222
1223
1224 # --- handler: friendly error for malformed min resolution -----
1225
1226
1227 async def test_handler_malformed_min_resolution_replies_help(

```



```

1228     memory_conn: aiosqlite.Connection, user: User, engine_holder: dict
1229 ) -> None:
1230     handler = make_config_command_handler(user, memory_conn, engine_holder)
1231     event = _make_mock_event("/set min resolution abc")
1232     await handler(event)
1233
1234     event.client.send_message.assert_called_once()
1235     args, _kwargs = event.client.send_message.call_args
1236     assert "Usage" in args[1]
1237     assert "720p" in args[1]
1238

```

9. user (2026-07-06T08:39:40.388Z)

```

618: # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
1634:async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1647:async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
1742:     "load_filter_config",
1743:     "save_filter_config",

```

10. assistant (2026-07-06T08:39:49.477Z)

The test_commands.py is unchanged and exercises the real handler end-to-end via mocked Telethon events. Now let me run a behavioral equivalence battery comparing old vs new. Let me focus on the two key divergence risks flagged: (1) the ``/set length`` / ``/set min length`` validation which old code re-loaded from DB but new code uses ``current_cfg``, and (2) the ``_update`` when no updates are applied.

Let me examine one critical divergence carefully. In the `**old**` code, ``/set length`` validation used ``current_cfg = await db.load_filter_config(conn, user.id)`` loaded fresh inside the branch. Then ``_update_config`` `**also**` reloaded from DB. In the `**new**` code, the adapter loads ``current_cfg`` once, passes it to ``handle()``, which uses it for validation AND ``_update`` uses ``current`` (the passed cfg) rather than reloading. Let me verify these are equivalent given no concurrent writes between the load and the update.

11. user (2026-07-06T08:39:50.627Z)

```

1634     async def load_filter_config(conn: aiosqlite.Connection, user_id: int) -> FilterConfig:
1635         """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1636
1637         ADR 0011 (Option 2A): ``filter_config`` is renamed to
1638         ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1639         instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1640         """
1641         policy = await get_default_policy(conn, user_id)
1642         if policy is None:
1643             return FilterConfig()
1644         return FilterConfig.model_validate_json(policy.config_json)
1645
1646
1647     async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1648         """Deprecated shim: upsert the account's default :class:`Policy`."""
1649         existing = await get_default_policy(conn, user_id)
1650         await upsert_policy(
1651             conn,
1652             Policy(
1653                 id=existing.id if existing is not None else None,
1654                 account_id=user_id,
1655                 name="default",
1656                 config_json=cfg.model_dump_json(),
1657                 is_default=True,
1658                 created_at=(existing.created_at if existing is not None else _now()),

```

```

1659         updated_at=_now(),
1660     ),
1661 )
1662
1663
1664 # --- bulk helpers (used by the manager on startup) -----
1665
1666
1667 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1668     conn: aiomysqlite.Connection, user_id: int, limit: int = 100
1669 ) -> list[VideoRecord]:
1670     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1671     conn.row_factory = aiomysqlite.Row
1672     async with conn.execute(
1673         """SELECT ci.*, s.provider_source_id AS src_pid

```

12. assistant (2026-07-06T08:39:53.051Z)

Now let me run a comprehensive equivalence battery. I'll implement the old handler logic and the new service, feed identical command strings, and diff both `reply\_text` and the saved config.

13. assistant (2026-07-06T08:40:15.102Z)

```
"""Equivalence battery: OLD commands.py handler vs NEW CommandService.handle.
```

We reconstruct the OLD handler by fetching main:src/tg_video_filter/commands.py into a temp module, and the NEW service from the working tree. Feed identical command strings through both and diff reply_text + saved config.

```

"""
import asyncio
import importlib.util
import subprocess
import sys
import types
from pathlib import Path

REPO = Path("C:/Users/avidu/Projects/Telegram")
sys.path.insert(0, str(REPO / "src"))

import aiomysqlite
from tg_video_filter import db
from tg_video_filter.models import FilterConfig, User
from tg_video_filter.command_service import CommandService

```

--- Load OLD commands.py from git main into a module ---

```

old_src = subprocess.run(
    ["git", "-C", str(REPO), "show", "main:src/tg_video_filter/commands.py"],
    capture_output=True, text=True, check=True,
).stdout
old_mod = types.ModuleType("old_commands")
old_mod.__dict__["__name__"] = "old_commands"
exec(compile(old_src, "old_commands.py", "exec"), old_mod.__dict__)

CMDS = [
    "/set size 100MB", "/set size 0MB", "/set size 1MB", "/set size 500KB",
    "/set size 1GB", "/set size 2TB", "/set size not-a-number", "/set size 10XB",
    "/set size 20MB", "/SET SIZE 10mb",
    "/set length 15m", "/set length 2s", "/set length 5s", "/set length 90",
    "/set length 10d", "/set max length 5m",
    "/set min length 5s", "/set min length 0s", "/set min length 120s", "/set min length 2m",
    "/set min resolution 720p", "/set min resolution 0p", "/set min resolution abc",
    "/set min res 480p", "/set min res xyz",
    "/set exclude gif on", "/set exclude round off", "/set exclude nosound on",
    "/set exclude gif true", "/set exclude unknown on", "/set exclude",

```

```

"/show config", "/show commands", "/show config ", "/show config extra",
"hello world", "", " /show config", "/set length 30s", # 30s vs custom min
]

```

Configs to test against (some commands are validated against current cfg)

```

BASE_CONFIGS = {
    "default": FilterConfig(),
    "min60": FilterConfig(min_duration_s=60),
    "max60": FilterConfig(max_duration_s=60),
    "max10": FilterConfig(max_duration_s=10),
}

USER_ID = 111

async def run_old(text, base_cfg):
    """Run old handler via a mock event; capture reply + saved config."""
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_user(conn, User(id=USER_ID))
    await db.save_filter_config(conn, USER_ID, base_cfg)
    engine_holder = {}
    replies = []

    class FakeClient:
        async def send_message(self, to, txt):
            replies.append(txt)

    class Peer:
        user_id = USER_ID

    class Msg:
        out = True
        peer_id = Peer()
        message = text

    class Event:
        message = Msg()
        client = FakeClient()

    # patch FilterEngine to a no-op so engine reload doesn't blow up
    import tg_video_filter.filters as f
    old_mod.__dict__["FilterEngine"] = f.FilterEngine
    old_mod.__dict__["db"] = db
    handler = old_mod.make_config_command_handler(User(id=USER_ID), conn, engine_holder)
    await handler(Event())
    saved = await db.load_filter_config(conn, USER_ID)
    await conn.close()
    return (replies[0] if replies else None), saved.model_dump()

async def run_new(text, base_cfg):
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_user(conn, User(id=USER_ID))
    await db.save_filter_config(conn, USER_ID, base_cfg)
    svc = CommandService()
    # adapter loads current_cfg fresh right before handle()
    current = await db.load_filter_config(conn, USER_ID)
    result = await svc.handle(USER_ID, text, conn, current)
    saved = await db.load_filter_config(conn, USER_ID)
    await conn.close()
    reply = result.reply_text if result.reply_text else None
    return reply, saved.model_dump()

```

```
async def main():
    diffs = []
    for cfg_name, base in BASE_CONFIGS.items():
        for text in CMDS:
            old_reply, old_cfg = await run_old(text, base)
            new_reply, new_cfg = await run_new(text, base)
            if old_reply != new_reply:
                diffs.append((cfg_name, text, "REPLY", repr(old_reply)[:80],
repr(new_reply)[:80]))
            if old_cfg != new_cfg:
                # find changed keys
                changed = {k: (old_cfg[k], new_cfg[k]) for k in old_cfg if old_cfg[k] !=
new_cfg[k]}
                diffs.append((cfg_name, text, "CONFIG", changed, ""))
    if not diffs:
        print("NO DIVERGENCE across", len(CMDS) * len(BASE_CONFIGS), "cases")
    else:
        print(f"{len(diffs)} DIVERGENCES:")
        for d in diffs:
            print(" ", d)

asyncio.run(main())
```

14. user (2026-07-06T08:40:15.126Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/equiv.py (file state is current in your context ? no need to Read it back)

15. user (2026-07-06T08:40:23.626Z)

NO DIVERGENCE across 156 cases

16. assistant (2026-07-06T08:40:32.919Z)

No behavioral divergence across 156 cases (39 commands × 4 base configs), covering reply strings AND saved config mutations. The refactor is behaviorally equivalent. Now let me focus on the assigned dimension: **test quality & coverage**.

Let me check coverage of the new files specifically and whether the new adapter's `on_command`` remains covered end-to-end.

17. user (2026-07-06T08:40:39.350Z)

.....

ERROR: Coverage failure: total of 52 is less than fail-under=80

[100%]

===== tests coverage =====

_____ coverage: platform win32, python 3.13.13-final-0 _____

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
src\tg_video_filter__init__.py	1	0	0	0	100%	
src\tg_video_filter__main__.py	7	7	0	0	0%	7-15
src\tg_video_filter\backfill.py	49	49	16	0	0%	17-405
src\tg_video_filter\bootstrap.py	23	23	0	0	0%	17-78
src\tg_video_filter\command_service.py	158	6	66	0	97%	210-211,
226-227, 246-247						
src\tg_video_filter\commands.py	44	0	10	0	100%	
src\tg_video_filter\config.py	26	5	2	0	75%	62-64, 69, 79
src\tg_video_filter\db.py	307	141	46	10	51%	195-196,
213-215, 292-312, 323, 343, 379-501, 520, 649-652, 683, 686, 788-791, 829, 844, 858, 893, 906, 919, 968-971, 1030-1039, 1085, 1097-1098, 1110-1139, 1146-1153, 1167-1194, 1204-1211, 1223-1229,						

```

1287-1314, 1321-1327, 1345-1368, 1375-1381, 1394-1406, 1411-1412, 1423-1428, 1449-1453,
1460-1467, 1492-1508, 1531-1534, 1545-1582, 1597, 1610, 1671-1681, 1749
src\tg_video_filter\delivery.py          102    102    48    0    0%    13-283
src\tg_video_filter\filters\__init__.py   39     10    18    4   65%    42-52, 63->65,
65->67, 67->69, 71->73
src\tg_video_filter\filters\base.py       26      2     0    0   92%    48, 52
src\tg_video_filter\filters\content_type.py 23     15    12    0   23%    18-22, 29-33,
40-44
src\tg_video_filter\filters\duration.py   15      8     6    0   33%    21-28
src\tg_video_filter\filters\resolution.py  13      7     6    0   32%    21-27
src\tg_video_filter\filters\size.py       18     10     6    0   33%    21-29, 34
src\tg_video_filter\manager.py            36     36     8    0    0%    26-204
src\tg_video_filter\models.py             162      7     8    3   93%    364, 366, 377,
416, 424-426
src\tg_video_filter\telethon_client.py    62     62    26    0    0%    16-358
-----
TOTAL                                1111    490    278    17   52%
FAIL Required test coverage of 80.0% not reached. Total coverage: 51.98%
226 passed in 3.43s

```

18. assistant (2026-07-06T08:40:45.015Z)

`commands.py` (the adapter) is at 100% coverage, exercised by test_commands.py's end-to-end handler tests. `command\_service.py` is at 97% with 6 uncovered lines: 210-211, 226-227, 246-247. Let me check what those are.

Lines 210-211, 226-227, 246-247 are the `except ValueError` branches for `parse\_size\_mb`/`parse\_duration\_s` in the size/length/min-length handlers. Let me verify.

19. user (2026-07-06T08:40:45.776Z)

```

205         m = _SIZE_RE.match(text)
206         if m:
207             value_str, unit = m.group(1), m.group(2) or ""
208             try:
209                 new_mb = parse_size_mb(value_str, unit)
210             except ValueError:
211                 return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
212             if new_mb < 1:
213                 return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
214             cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
215             return CommandResult(
216                 f"? Max file size set to
**{_format_size(new_mb)}**.\n\n{format_config(cfg)}",
217                 config_changed=True,
218             )
219
220         # --- /set length <value><unit> -----
221         m = _LENGTH_RE.match(text)
222         if m:
223             value_str, unit = m.group(1), m.group(2) or ""
224             try:
225                 new_secs = parse_duration_s(value_str, unit)
226             except ValueError:
227                 return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
228             min_secs = current_cfg.min_duration_s or 0
229             if new_secs < min_secs:
230                 return CommandResult(
231                     f"? Max duration ({_format_duration(new_secs)}) must be "
232                     f">= min duration ({_format_duration(min_secs)})."
233                 )
234             cfg = await self._update(conn, user_id, current_cfg,

```

```

max_duration_s=new_secs)
235         return CommandResult(
236             f"? Max duration set to
**[_format_duration(new_secs)]**.\\n\\n{format_config(cfg)}",
237             config_changed=True,
238         )
239
240     # --- /set min length <value><unit> -----
241     m = _MIN_LENGTH_RE.match(text)
242     if m:
243         value_str, unit = m.group(1), m.group(2) or ""
244         try:
245             new_min = parse_duration_s(value_str, unit)
246         except ValueError:
247             return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
248         max_secs = current_cfg.max_duration_s or 0
249         if max_secs > 0 and new_min > max_secs:
250             return CommandResult(
251                 f"? Min duration ({_format_duration(new_min)}) must be "
252                 f"<= max duration ({_format_duration(max_secs)})."
253             )
254         cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)

```

20. assistant (2026-07-06T08:40:55.774Z)

Confirmed. The 6 uncovered lines (210-211, 226-227, 246-247) are the `except ValueError` branches. These branches are actually **unreachable in practice**: the regexes `\_SIZE\_RE`/`\_LENGTH\_RE`/`\_MIN\_LENGTH\_RE` only permit unit characters `[kmg]?b?` / `[smh]?` that `parse\_size\_mb`/`parse\_duration\_s` always accept, and the numeric group is `\\d+` which `int()` always parses. So `parse\_\*` can never raise `ValueError` for a string the regex matched. This is defensive dead code ? pre-existing in the old file too (old code had the same branches, also untested). Not a new defect; just a coverage note.

Now, the key test-quality question flagged: **does test_command_service.py assert actual reply strings + config mutations, or just smoke-test?** Let me assess a specific gap. Note test line 97-99:

```

`test_set_size_invalid_format_returns_empty` sends `/set size not-a-number` and asserts
`reply_text == ""`. But this input doesn't match the regex ? so it tests the noise fallback,
NOT the `except ValueError` branch. The test's docstring even says "regex doesn't match". So the
ValueError branches remain untested, and there's a misleadingly-named test implying it
covers invalid-format handling when it actually covers the unrecognized-command path.

```

Let me verify the config-mutation assertions are real (not just smoke). Let me check whether any test actually asserts the persisted config value after a successful mutation in test_command_service.py.

21. user (2026-07-06T08:40:57.950Z)

(Bash completed with no output)

22. assistant (2026-07-06T08:41:05.558Z)

Confirmed a real test-quality gap. **None** of the 20 tests in `test\_command\_service.py` load the config back from the DB to assert the actual persisted mutation. They only assert `result.config\_changed is True` and substring-check the reply. So the service's core job ? persisting the correct field/value via `\_update` + `db.save\_filter\_config` ? is **not directly verified** in the new test file.

Now, is `\_update` actually covered? The end-to-end persistence IS exercised through `test\_commands.py` (e.g. line 541 `loaded.max\_size\_mb == 512`), because the adapter calls `svc.handle` ? `svc.\_update`. So coverage doesn't drop and behavior is guarded. But test_command_service.py itself doesn't independently verify persistence ? a regression in

`\_update`'s field mapping (e.g., swapping `exclude\_gif`?`exclude\_round`) would pass all 20 service tests while being caught only by test_commands.py.

Let me verify that claim: could a field-mapping swap in `\_update` pass all of test_command_service.py?

23. user (2026-07-06T08:41:09.884Z)

Reply label is decoupled from field_map -> swap not caught by service tests

24. assistant (2026-07-06T08:41:17.844Z)

Now let me confirm the re-export shim question: does any test_commands.py assertion pass only because of a re-export shim rather than real behavior? The re-exported symbols (`\_SIZE\_RE`, `parse\_size\_mb`, etc.) are the *same objects* from command_service, and `\_update\_config` is a real delegating wrapper that hits the DB. The regex/parser tests test real objects; the `\_update\_config` tests (lines 400-434) exercise real DB persistence through the wrapper. So no test passes on a hollow shim. Good.

Let me also confirm the adapter's `config\_changed` ? double-load path is covered and check the one behavioral subtlety: the new adapter reloads config from DB after a change (line 74) rather than using the returned cfg. Verify engine gets the right config.

25. user (2026-07-06T08:41:23.046Z)

```
src\tg_video_filter\telethon_client.py          62      62      26      0      0%    16-358
-----
TOTAL                                           1111    491    278     18    52%
FAIL Required test coverage of 80.0% not reached. Total coverage: 51.84%
198 passed in 2.67s
```

26. assistant (2026-07-06T08:41:29.601Z)

All 198 test_commands.py tests pass (the coverage FAIL is just because I ran a subset). The adapter's engine-reload path (line 74-75) is covered by `test\_handler\_set\_size\_persists\_and\_replies` (asserts `engine\_holder[ENGINE\_KEY].cfg.max\_size\_mb == 512`) and others.

I have a complete picture. Let me summarize my findings on the test quality & coverage dimension. The refactor is behaviorally equivalent (156/156 cases identical). The findings are all test-quality gaps ? no behavioral defects, no lost functionality, no broken re-exports, no coverage regression (adapter 100%, service 97%).

27. user (2026-07-06T08:41:47.976Z)

Structured output provided successfully